

Developer Handbook

Version 2

NETZSCH AI Gateway — Integration Guide for Applications and AI Coding Agents

This handbook explains how NETZSCH developers can use the centralized NETZSCH AI Models in development workflows with AI coding agents such as Codex and Claude Code.

In this PDF, underscores such as “_” may not be displayed correctly at certain zoom levels. To avoid misreading commands, file names, or configuration values, make sure to use a sufficiently high zoom level, because underscores may otherwise appear incomplete or disappear depending on the PDF rendering.

It covers two areas:

- integrating the NETZSCH AI Gateway into your own applications via the OpenAI-compatible API
- integrating an AI coding agent (Codex or Claude Code) into your development workflow with a recommended configuration baseline

Scope and Responsibility

This handbook is a guide for integrating NETZSCH AI Gateway (LiteLLM) API keys into applications and AI coding agents. The configurations shown here are recommendations that reflect a security-conscious baseline — they are not a NETZSCH-wide approval or endorsement of Codex, Claude Code, or any other tool.

Whether and how AI coding agents are used is decided within each team. Every team defines its own approved tools, configurations, and working agreements, and every user must follow the guidelines of their own team. If you are unsure which rules apply to you, clarify this with your team before setting up an agent.

Users remain responsible for their own use of these AI coding agents. Even with a carefully chosen configuration, every generated command, file change, tool call, approval prompt, and external access request must still be reviewed carefully before approval or execution.

If it is not clear what a command or tool request will do, stop and reject or cancel it. Deleted or modified files may not be recoverable by IT in every case.

Table of Contents

- Scope and Responsibility1
- Prerequisites3
- Currently Available Models3
- Integrating AI into your application4
 - Using the API4
 - Endpoint URL4
 - Authentication4
 - Code Examples4
- Integrating AI into your Development Environment.....6
 - Security Implications6
 - Safe Operating Checklist.....6
 - Codex (CLI & IDE).....7
 - Install the Codex CLI7
 - Install the Codex IDE Extension.....7
 - Configure Codex for NETZSCH7
 - Meaning of the config.toml settings9
 - Use Codex in VS Code10
 - Claude Code (CLI & IDE).....11
 - IDE Integration13
- Rate Limits and Usage Limits.....15
 - Rate Limits15
 - Budget Limits15
 - Check your Balance15

Prerequisites

To use the provided NETZSCH AI Models, you need to request a valid API key from the NETZSCH Business Services Team via a service request.

This API key may only be used for development and testing purposes. If an API key is required for production application usage, a separate service request must be submitted to obtain an Application Key.

Please also make sure that your IDE, the Codex CLI, and the Codex extension are up to date. Older versions can lead to setup problems, missing features, or unexpected behavior.

If you need a new installation or an update of required software on your corporate device, this is typically handled by the IT Operations team via the normal IT ticket process.

Afterwards, there are two paths you can take with your API key:

- integrate the API into any application such as a RAG pipeline or chatbot
- integrate the API into an AI coding agent (Codex or Claude Code) for CLI- and IDE-based development workflows

Currently Available Models

The following models are currently available through the NETZSCH AI Gateway.

Interactive models for API usage and Codex model selection

- gpt-5
- gpt-5.1
- gpt-5-mini
- gpt-5-nano
- gpt-5.4
- gpt-5.5

Additional interactive models currently available through the NETZSCH AI Gateway

- gemini-3.1-flash-lite
- gemini-3.5-flash
- claude-sonnet-4-6
- claude-sonnet-5
- claude-opus-4-8
- claude-fable-5
- claude-haiku-4-5

Embedding models for API integrations

- text-embedding-large
- text-embedding-small

The GPT models in the first list are the relevant choices for Codex CLI and the Codex IDE extension. The Claude and Gemini models are available through the NETZSCH AI Gateway for other supported integration scenarios. The embedding models are intended for retrieval, search, and similar API-based embedding use cases and are not Codex conversation models.

Integrating AI into your application

Together with your API key, the API URL `https://litellm.chatbot.netzsch.com` acts OpenAI-API-compatible. That means it can be used as a drop-in replacement for many applications that already use AI integrations.

Using the API

Endpoint URL

This is the endpoint URL for all models:

```
https://litellm.chatbot.netzsch.com/
```

Authentication

Include your API key in all requests. The API key always starts with `sk-`.

Most SDKs will handle authentication for you. If not, add your key to your request header like this:

```
Authorization: Bearer sk-...
```

You can get a live overview of the currently available models and their pricing with this cURL request:

```
curl https://litellm.chatbot.netzsch.com/models \
-H "Authorization: Bearer sk-..."
```

Code Examples

TypeScript

You can install the official OpenAI package via npm, bun, yarn, or similar tooling with `npm install openai` and use it like this:

```
import OpenAI from "openai";

async function main() {
  const client = new OpenAI({
    apiKey: "sk-...",
    baseUrl: "https://litellm.chatbot.netzsch.com/",
  });

  try {
    const response = await client.chat.completions.create({
```

```
    model: "gpt-5.1",
    messages: [
      {
        role: "user",
        content: "this is a test request, write a short poem",
      },
    ],
  });

  console.log(response);
  console.log("Generated poem:");
  console.log(response.choices[0].message.content);
} catch (error) {
  console.error("Error:", error);
}
}

main();
```

Python

Install the official OpenAI SDK from PyPI with `pip install openai`:

```
import openai

client = openai.OpenAI(
    api_key="sk-...",
    base_url="https://litellm.chatbot.netzsch.com/"
)

response = client.chat.completions.create(
    model="gpt-5.1",
    messages=[
        {
            "role": "user",
            "content": "this is a test request, write a short poem"
        }
    ]
)

print(response)
```

Integrating AI into your Development Environment

To use AI efficiently for agentic engineering, you can integrate an AI coding agent directly into your command line and IDE. This handbook documents the gateway integration for two commonly used agents: Codex and Claude Code. Which agents your team uses — and with which settings — is defined by your team; this section provides recommended baselines and integration steps. The security principles and the safe operating checklist below apply to any agent; the tool-specific setup follows in the respective sections.

Security Implications

These security principles apply to any AI coding agent you run at NETZSCH, including Codex and Claude Code. They are recommendations from the platform team — your team's own guidelines take precedence and may be stricter.

AI coding agents can read files, write files, run commands, and use external capabilities depending on the active permission profile and enabled integrations. Without proper sandboxing and review, that can lead to data loss, credential exposure, unwanted network access, or changes in the wrong repository.

Keep the following security principles in mind:

- Use the recommended configuration baselines shown in this handbook as a starting point and align your final setup with your team's guidelines.
- Keep access limited to the current workspace and review every approval prompt before accepting it.
- Do not trust unfamiliar repositories casually. Project-local `.codex/config.toml`, project instructions, hooks, and rules can change Codex behavior for trusted projects.
- Be careful with MCP servers, plugins, browser/computer-use style integrations, and other connectors, and only enable them in line with your team's guidelines. These integrations expand what the agent can access.
- Sandboxing command execution is not the same as disabling web search. Web search can still expose retrieved content to the model, so we recommend keeping it disabled for routine work or using it only when explicitly required, reviewed, and in line with your team's guidelines.
- AI coding agents are susceptible to prompt injection. Malicious instructions can be hidden in source files, websites, issues, documentation, or generated content and may try to trigger destructive local actions or exfiltration attempts.
- Treat `CODEX_HOME` and `~/ .codex` as locally stored sensitive developer data. Codex can keep local history, logs, and cache data there, so prompts, snippets, file paths, and similar context should not be treated as disposable.

Permission profiles in Codex are currently a beta feature. Future updates to this page must keep using the recommended permission-profile approach and must not mix it with older legacy examples such as `sandbox_mode` or `sandbox_workspace_write`.

Safe Operating Checklist

Before you start a session with Codex or Claude Code, verify the following:

- Open only the repository you actually intend to work on.

- Confirm that the active access level shown by `/permissions` matches the configuration recommended here or the baseline defined by your team.
- Confirm that filesystem access is workspace-scoped and not broader than required.
- Review every proposed command, file edit, approval prompt, and connector request before allowing it.
- Stop immediately if a command, tool request, or access request is unclear.

Codex (CLI & IDE)

Codex is the official CLI and TUI by OpenAI and can also be used through the Visual Studio Code extension.

At this time, Codex is fully supported only with GPT models. Other models may work with limitations, but they are not officially supported.

Software prerequisites

Before setting up Codex, make sure the following software is available on your device:

- Node.js including `npm`, if you want to install the Codex CLI via `npm`
- Homebrew, if you want to install the Codex CLI on macOS via `brew`
- a terminal environment such as PowerShell, Windows Terminal, Command Prompt, or a macOS/Linux shell
- Visual Studio Code, if you want to use the Codex extension in your IDE

If one of these components is missing on your corporate device, request the installation through the normal IT process first.

Install the Codex CLI

You can install the Codex CLI via `npm` or Homebrew:

```
npm install -g @openai/codex
```

```
# Homebrew on macOS  
brew install --cask codex
```

After the installation, you should be able to start Codex from a terminal with:

```
codex
```

Install the Codex IDE Extension

If you want to use Codex inside Visual Studio Code, install the Codex extension from the VS Code Extension Marketplace.

You can do this in one of the following ways:

- open the Extensions view in VS Code, search for `OpenAI`, and install the Codex extension

- open the Visual Studio Code Marketplace in your browser and install the extension from there

If extension installation from the marketplace is restricted on your managed device, request access or installation through IT.

Configure Codex for NETZSCH

The recommended NETZSCH setup for Codex has two parts:

- Set the `NETZSCH_LLM_API_KEY` environment variable so Codex can authenticate against the NETZSCH gateway.
- Configure `~/.codex/config.toml` with the NETZSCH provider and the recommended permission profile.

First, expose the required environment variable.

On macOS or Linux shells:

```
export NETZSCH_LLM_API_KEY="sk-..."
```

On Windows PowerShell:

```
setx NETZSCH_LLM_API_KEY "sk-..."
```

After setting the environment variable on Windows, restart all open Command Prompt or terminal windows and your IDEs such as VS Code so the new value is picked up.

Then edit `~/.codex/config.toml`:

- on Windows, for example `C:\Users\your.user\.codex\config.toml`
- on macOS or Linux, for example `~/.codex/config.toml`

If this is your first Codex setup, the file may not exist yet. In that case, create `config.toml` manually once in the `.codex` directory and then paste in the configuration below.

If `config.toml` already exists, update it so Codex uses the NETZSCH provider setup and the recommended security configuration shown below.

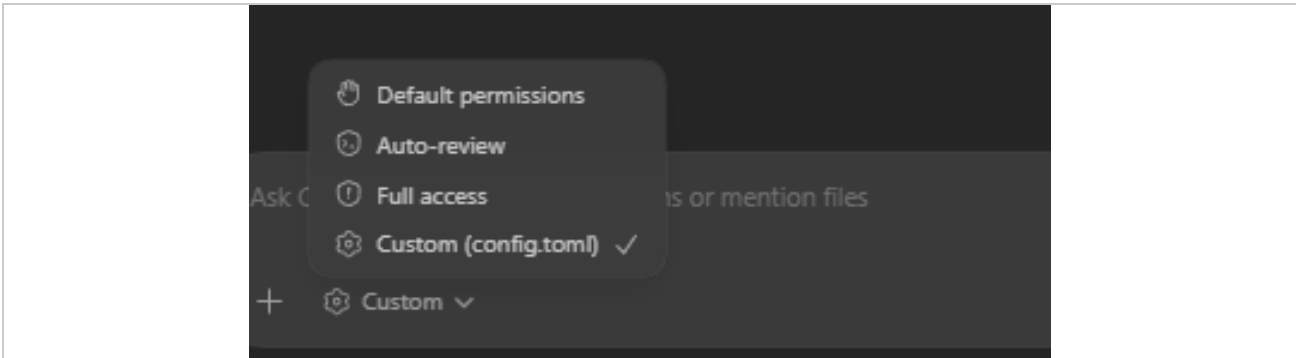
```
model = "gpt-5.4"
model_provider = "netzsch"
model_reasoning_effort = "medium"
default_permissions = ":workspace"
approval_policy = "untrusted" # recommended alternative: "on-request"

[model_providers.netzsch]
name = "NETZSCH"
base_url = "https://litellm.chatbot.netzsch.com/v1"
env_key = "NETZSCH_LLM_API_KEY"
wire_api = "responses"

[windows]
sandbox = "unelevated" # alternative: "elevated" (recommended with local admin rights)
```


We recommend the approval policy values `untrusted` or `on-request`, unless your team defines a different baseline. The choice is up to the user and the team. `untrusted` is the stricter baseline. `on-request` is the less restrictive option if you want Codex to work more smoothly inside the current workspace while still prompting for confirmation when an action needs escalation.

With `approval_policy = "on-request"` and a workspace-write setup, Codex can usually read files, edit files, and run commands inside the working directory automatically. It will still ask for approval when it needs to go beyond the workspace boundary or otherwise escalate, for example for edits outside the workspace or commands that require network access. `approval_policy = "untrusted"` keeps the more restrictive recommended baseline.



Recommended Codex configuration level

Meaning of the config.toml settings

- `model = "gpt-5.4"` defines which model Codex should use by default for requests in the recommended baseline.
- `model_provider = "netzsich"` tells Codex to use the provider configuration stored under `[model_providers.netzsich]`.
- `model_reasoning_effort = "medium"` sets the reasoning depth Codex should use for this model.
- `default_permissions = ":workspace"` uses the built-in workspace permission profile. It allows Codex to work inside the current workspace while keeping access to other locations more restricted.
- `approval_policy = "untrusted"` is the stricter option and keeps the more restrictive approval behavior.
- `approval_policy = "on-request"` is the recommended alternative. In a workspace-write setup, Codex can usually work inside the current workspace without repeated confirmation, but it still asks when an action needs escalation, for example outside the workspace or for network-requiring commands.
- `[model_providers.netzsich]` starts the provider definition for the NETZSCH gateway connection.
- `name = "NETZSCH"` sets the display name of this provider profile.
- `base_url = "https://litellm.chatbot.netzsich.com/v1"` defines the API endpoint Codex sends its model requests to.
- `env_key = "NETZSCH_LLM_API_KEY"` tells Codex to read the API key from the environment variable with this name.
- `wire_api = "responses"` defines which API format Codex uses when talking to the configured provider.
- `[windows]` starts the Windows-specific runtime settings.

- `sandbox = "unelevated"` is the fallback Windows sandbox mode when local admin rights are not available or elevated setup is blocked.
- `sandbox = "elevated"` is the recommended Windows sandbox mode when local admin rights are available, because it provides stronger sandbox isolation on native Windows.

With the above `config.toml`, Codex uses the NETZSCH provider, reads the API key from `NETZSCH_LLM_API_KEY`, uses the built-in `:workspace` permission profile by default, and keeps Windows execution in a native sandbox. If available, `elevated` is preferred over `unelevated`.

Read more about the Codex configuration basics (developers.openai.com/codex/config-basic), permissions (developers.openai.com/codex/permissions), and sandboxing (developers.openai.com/codex/agent-approvals-security).

Use Codex in VS Code

The Codex VS Code extension uses the Codex CLI and shares the same local `~/ .codex/config.toml`. Do not create or rely on a separate IDE-only security profile.

Model selection also follows this shared setup: the Codex CLI can switch between available conversation models via `/model` or `--model`, and the IDE extension uses the same shared Codex model layer rather than a separate IDE-only model catalog.

If you use Codex in VS Code:

- make sure the extension points to the same local Codex installation that you use in the terminal
- open the Codex settings UI if you need to inspect which CLI and config file the extension is using
- open the shared `config.toml` from the Codex settings UI when you need to review or update the NETZSCH configuration

Codex also shows the active access level via `/permissions`. We recommend keeping this on the configuration shown above, unless your team defines a different baseline. Be aware that switching Codex to **Full Access** or to a less restrictive configuration removes the intended safeguards and may give the model much broader access to your computer — check your team's guidelines before doing so.

You are now ready to start Codex:

```
codex
```

Claude Code (CLI & IDE)

Claude Code is a developers favorite, however, it is known to burn the most tokens in the shortest amount of time, costing the most money.

It is therefore most relevant when you specifically want to work with Claude models, which are available through the NETZSCH AI Gateway.

There are multiple ways to install Claude Code, however the preferred way is the native Installation for macOS and WSL:

```
curl -fsSL https://claude.ai/install.sh | bash
```

however, PowerShell is also possible:

```
irm https://claude.ai/install.ps1 | iex
```

You can find detailed Installation Instructions here (code.claude.com/docs).

On Windows, Claude Code sandboxing is currently only supported when Claude Code runs inside WSL. Running Claude Code directly in native Windows without sandboxing is discouraged — if your team allows a native Windows setup, be aware that the sandbox protections described below will not apply.

Afterwards, create a `~/.claude/settings.json` file with the recommended baseline:

```
{
  "$schema": "https://json-schema.org/claude-code-settings.json",
  "env": {
    "ANTHROPIC_BASE_URL": "https://litellm.chatbot.netzsch.com",
    "ANTHROPIC_AUTH_TOKEN": "sk-...",

    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "claude-haiku-4.5",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "claude-sonnet-4.6",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "claude-opus-4.6",

    "CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS": "1",
    "CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC": "true",
    "CLAUDE_CODE_DISABLE_TELEMETRY": "1",
    "DISABLE_TELEMETRY": "1"
  },

  "sandbox": {
    "enabled": true,
    "allowUnsandboxedCommands": false,
    "network": {
      "allowedDomains": [
        "litellm.chatbot.netzsch.com",
        "*.netzsch.com",
        "*.azure.com"
      ]
    }
  },

  "permissions": {
```

```
"deny": ["Read(./env)", "Read(./env.*)", "Read(~/.ssh/**)"]
}
```

This file makes sure that Claude Code connects to the NETZSCH Internal AI Gateway, sets three different base models (via `ANTHROPIC_DEFAULT_HAIKU_MODEL` etc.) which you can later on switch out for newer models:

- Haiku is small, fast and cheap
- Sonnet is the standard middleground
- Opus is really smart but also extremely cost-extensive

Note on Claude Fable 5: Claude Code only knows the three model tiers Haiku, Sonnet, and Opus in `settings.json` — there is no `ANTHROPIC_DEFAULT_FABLE_MODEL` entry. The only way to switch to Fable 5 (available through the NETZSCH AI Gateway) is by running the following command inside a Claude Code session:

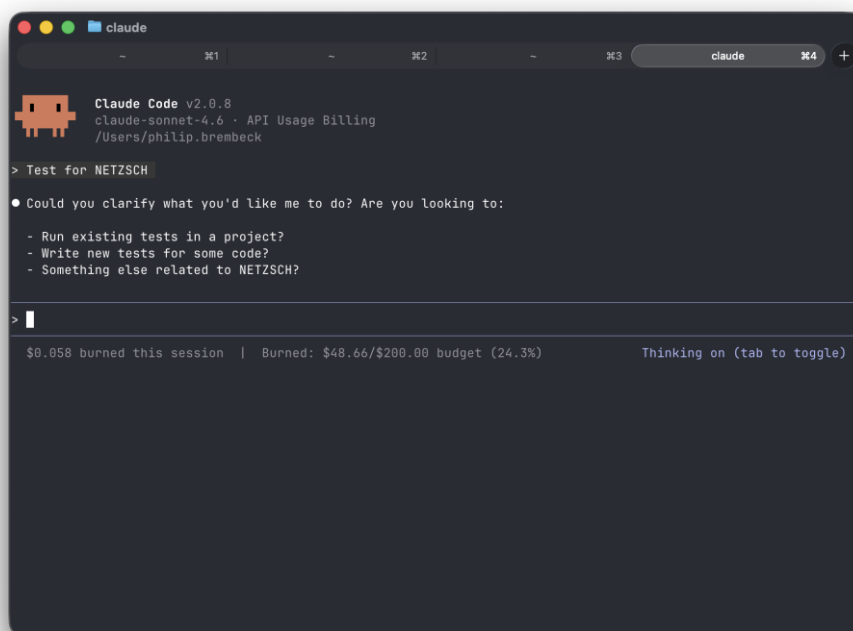
```
/model claude-fable-5
```

This also saves Fable 5 as the default model for new sessions. Be aware that Fable 5 is priced significantly above Opus, so use it deliberately.

The config then disables Beta features, which are not supported by our Gateway, turns off any kind of non-essential traffic (telemetry data, bug reports, autoupdate, etc.) and enables a Sandbox with the following permissions:

- `allowUnsandboxedCommands = false`: Commands can only be executed in a sandbox-environment. Your system is never at risk.
- Explicitly allows only certain domains for Network traffic, curl, etc. (our Gateway-URL has to be in there as well)
- Explicitly denies Read-Permissions (gets inherited by Write) for `.env` files and the `.ssh`-folder.

On Windows, this sandboxed setup currently means using Claude Code inside WSL. A native Windows setup without sandbox protection is not recommended. You can adapt this sandbox configuration to your team's needs. You can then invoke Claude Code by typing `claude` in your terminal.

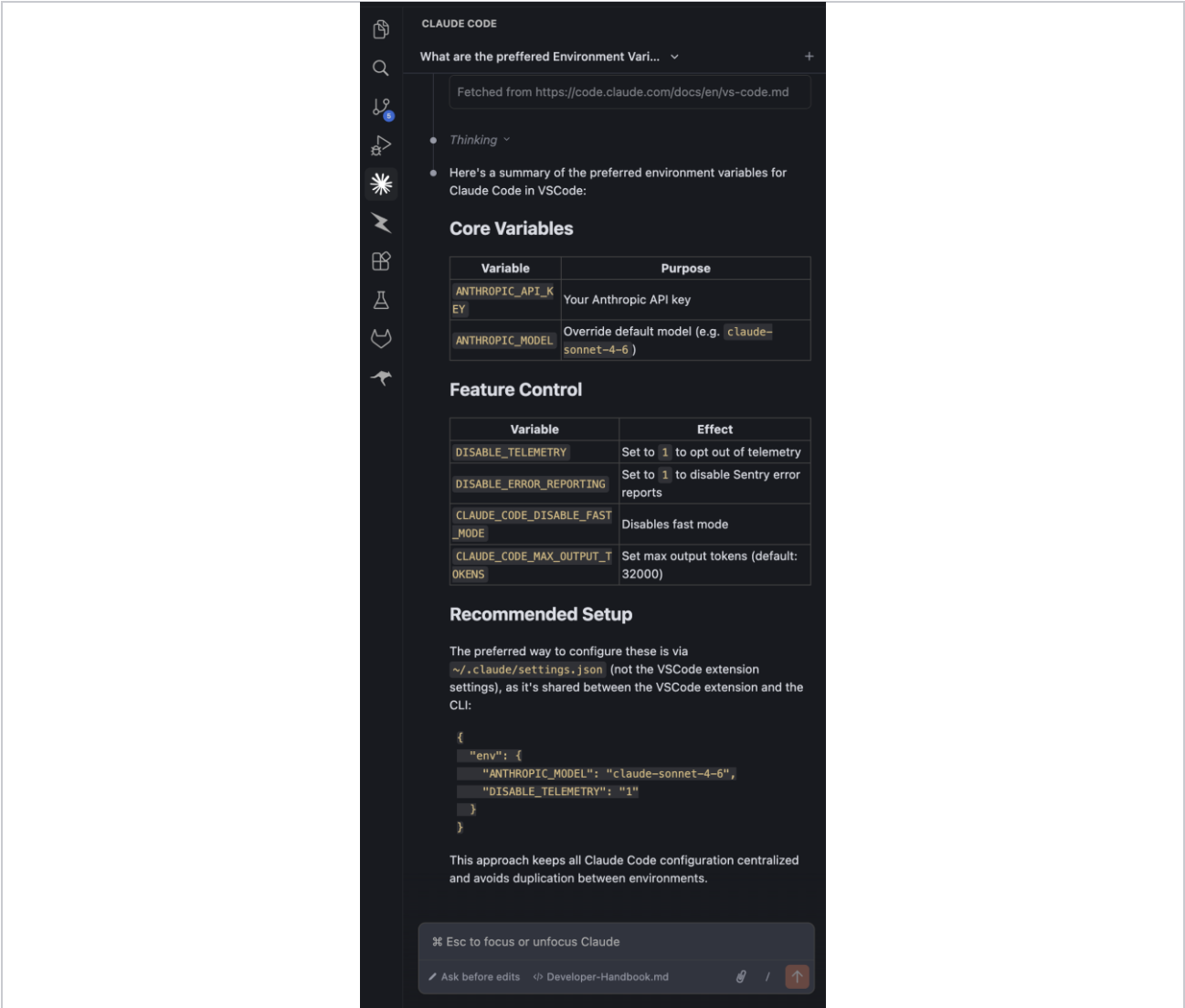
*Claude Code CLI*

Claude Code also offers an IDE Extension for VSCode. It can be downloaded from here (marketplace.cursorapi.com).

The IDE Integration then also needs to be setup, if its not working directly. Press `cmd+shift+p` or `win+shift+p` in VSCode to Open the Command Pallet. Then, type in `Open User Settings (JSON)`. Scroll to the bottom of this file and add the following:

```
"claudeCode.selectedModel": "claude-sonnet-4.6",
"claudeCode.environmentVariables": [
  {"name": "ANTHROPIC_AUTH_TOKEN", "value": "sk..."},
  {"name": "ANTHROPIC_BASE_URL", "value": "https://litellm.chatbot.netzsch.com"},
  {"name": "ANTHROPIC_MODEL", "value": "claude-sonnet-4.6"},
  {"name": "ANTHROPIC_DEFAULT_HAIKU_MODEL", "value": "claude-haiku-4.5"},
  {"name": "DISABLE_TELEMETRY", "value": "1"},
  {"name": "CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC", "value": "true"}
]
```

afterwards, Claude Code can be used within VSCode:



Claude Code IDE

Rate Limits and Usage Limits

Rate Limits

Requests per Minute

Requests are rate limited to a maximum of 100 requests per minute. This should usually be enough for agentic engineering, but it can become a limitation for application integrations. Every request above 100 will be rejected by the gateway until enough of the rate-limiting window has cleared.

This limit can be manually increased. If this is a problem for a project, please talk to the NBS Team.

Tokens per Minute

The maximum tokens per minute are also rate limited to 500,000 tokens, so that providers, the AI gateway, and shared limits are not overwhelmed. This limit is currently not intended to be increased, but that may change in the future.

Budget Limits

Budgets are individually communicated. They can reset after a specified time period, in most cases every 30 days. If a budget is used up within its current period, the API key will stop working. You can talk to the NBS team to request an increase.

Check your Balance

For a quick balance check, call the key info endpoint directly with your API key.

Cross-platform cURL example

```
curl -s https://litellm.chatbot.netzsch.com/key/info \
-H "Authorization: Bearer sk-..."
```

PowerShell example

```
Invoke-RestMethod -Uri "https://litellm.chatbot.netzsch.com/key/info" `
-Headers @{ Authorization = "Bearer sk-..." }
```

If you want to build a more comfortable local helper command or alias, keep in mind that the previous long alias example was Bash-oriented and not suitable for Windows PowerShell without adaptation.